

Министерство науки и высшего образования Российской Федерации  
Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
"Белгородский государственный технологический университет им. В. Г.  
Шухова"  
(БГТУ им. В.Г. Шухова)



Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники  
и автоматизированных систем

**Лабораторная работа № 2**  
**по дисциплине: Вычислительная математика**  
**по теме: Алгебра матриц. Быстрое умножение матриц.**  
**Вычисление обратной матрицы. Нахождение собственных**  
**чисел и собственных векторов матрицы.**

Выполнил: студент группы ПВ-221  
Дорошенко Владимир Юрьевич  
Проверил: ст. преподаватель:  
Четвертухин В.Р.

Белгород 2024

**Цель работы: Изучить алгебраические операции над матрицами, особенности алгоритмизации быстрых матричных алгоритмов (на примере умножения матриц), вычисления обратной матрицы, нахождения собственных чисел и собственных векторов матрицы.**

*Первая часть* данного задания предполагает нахождение обратной матрицы вручную по алгоритму, спецификация которого приводится в приложении к данной лабораторной работе. Для ручной проверки корректности полученного результата предлагается выполнить решение СЛАУ из лабораторной работы No1 методом обратной матрицы.

*Вторая часть* задания предполагает написание и выполнение в интерактивном блокноте Jupyter коротких программ на языке Python для: а) нахождения обратной матрицы (использовать данные своего индивидуального задания); б) нахождения собственных чисел и собственных векторов матрицы (использовать те же данные).

Ручной метод нахождения  
обратной матрицы

$$\left( \begin{array}{ccc|ccc} 185 & -174 & 208 & 1 & 0 & 0 \\ 225 & -193 & 237 & 0 & 1 & 0 \\ -217 & 190 & -239 & 0 & 0 & 1 \end{array} \right)$$

$$\left( \begin{array}{ccc|ccc} 1 & 0,94 & 1,12 & 0,05 & 0 & 0 \\ 225 & -193 & 237 & 0 & 1 & 0 \\ -217 & 190 & -239 & 0 & 0 & 1 \end{array} \right)$$

$$\left( \begin{array}{ccc|ccc} 1 & 0,94 & 1,12 & 0,05 & 0 & 0 \\ 0 & 40,5 & 15 & 11,25 & 1 & 0 \\ 0 & 393,98 & 4,04 & 10,85 & 0 & 1 \end{array} \right)$$

$$\left( \begin{array}{ccc|ccc} 1 & 0,94 & 1,12 & 0,05 & 0 & 0 \\ 0 & 1 & 0,037 & 0,278 & 0,024 & 0 \\ 0 & 393,98 & 4,04 & 10,85 & 0 & 1 \end{array} \right)$$

$$\left( \begin{array}{ccc|ccc} 1 & 0,94 & 1,12 & 0,05 & 0,0220 & \\ 0 & 1 & 0,037 & 0,0278 & 0,024 & 0 \\ 0 & 393,98 & 4,04 & 10,85 & 0 & 1 \end{array} \right) \begin{array}{l} \text{I} \\ \\ \times 0,94 \end{array}$$

$$\left( \begin{array}{ccc|ccc} 1 & 0 & -1,085 & -0,0238 & 0,0220 & \\ 0 & 1 & 0,037 & 0,0278 & 0,024 & 0 \\ 0 & 393,98 & 4,04 & 10,85 & 0 & 1 \end{array} \right) \begin{array}{l} \\ \times 393,98 \\ \text{III} \end{array}$$

$$\left( \begin{array}{ccc|ccc} 1 & 0 & -1,085 & -0,0238 & 0,0220 & \\ 0 & 1 & 0,037 & 0,0278 & 0,024 & 0 \\ 0 & 0 & 10,537 & 0,103 & 9,45 & 1 \end{array} \right) : 10,537$$

$$\left( \begin{array}{ccc|ccc} 1 & 0 & -1,085 & -0,0238 & 0,02256 & 0 \\ 0 & 1 & 0,037 & 0,0278 & 0,024 & 0 \\ 0 & 0 & 1 & 0,0097 & 0,896 & 1 \end{array} \right) \begin{array}{l} \\ \text{II} \\ \times 0,037 \end{array}$$

$$\left( \begin{array}{ccc|ccc} 1 & 0 & -1,085 & -0,0238 & 0,02256 & 0 \\ 0 & 1 & 0 & -0,0244 & 0,00915 & 0,095 \\ 0 & 0 & 1 & 0,0097 & 0,896 & 0,095 \end{array} \right) \begin{array}{l} \\ \text{I}_+ \\ \times -1,085 \end{array}$$

$$\left( \begin{array}{ccc|ccc} 1 & 0 & 0 & -0,01127 & 0,0121 & 0,000249 \\ 0 & 1 & 0 & -0,001296 & 0,00404 & -0,00588 \\ 0 & 0 & 1 & 0,00176 & -0,00628 & -0,0066 \end{array} \right)$$

Übungen:

$A^{-1} =$

$$\begin{pmatrix} -0,01227 & 0,012 & 0,00025 \\ -0,00129 & 0,00404 & -0,0058 \\ 0,00176 & -0,00629 & -0,0068 \end{pmatrix}$$

## Содержимое блокнота

"Программа, демонстрирующая алгоритмическую реализацию умножения Штрассена для матриц на языке Python"

```
[ ] import numpy as np

def split_matrix(matrix):
    row, col = matrix.shape
    row2, col2 = row // 2, col // 2
    return (
        matrix[:row2, :col2], matrix[:row2, col2:],
        matrix[row2:, :col2], matrix[row2:, col2:]
    )

def strassen_multiply(matrix1, matrix2):
    if len(matrix1) == 1:
        return matrix1 * matrix2

    a, b, c, d = split_matrix(matrix1)
    e, f, g, h = split_matrix(matrix2)

    p1 = strassen_multiply(a, f - h)
    p2 = strassen_multiply(a + b, h)
    p3 = strassen_multiply(c + d, e)
    p4 = strassen_multiply(d, g - e)
    p5 = strassen_multiply(a + d, e + h)
    p6 = strassen_multiply(b - d, g + h)
    p7 = strassen_multiply(a - c, e + f)

    result_top_left = p5 + p4 - p2 + p6
    result_top_right = p1 + p2
    result_bottom_left = p3 + p4
    result_bottom_right = p1 + p5 - p3 - p7

    top_half = np.hstack((result_top_left, result_top_right))
    bottom_half = np.hstack((result_bottom_left, result_bottom_right))
    result = np.vstack((top_half, bottom_half))

    return result
```

+ Код

+ Текст

"Код для «прямого» перемножения матриц, чтобы свериться с правильностью кода для метода Штрассена"

```
[ ] def matrix_multiply(matrix1, matrix2):
    if len(matrix1[0]) != len(matrix2):
        raise ValueError("Несовместимые матрицы для умножения")

    result = [[0 for _ in range(len(matrix2[0]))] for _ in range(len(matrix1))]

    for i in range(len(matrix1)):
        for j in range(len(matrix2[0])):
            for k in range(len(matrix2)):
                result[i][j] += matrix1[i][k] * matrix2[k][j]

    return result
```

Программа, которая генерирует квадратную матрицу размером n x n из случайных чисел.

Параметры:

n (int): Размерность матрицы (n x n).

min\_val (int): Минимальное значение для элементов матрицы (включительно).

max\_val (int): Максимальное значение для элементов матрицы (включительно).

Возвращает:

np.array: Сгенерированная матрица размером n x n.

```
[ ] def generate_square_matrix(n, min_val=0, max_val=10):
    matrix = np.random.randint(min_val, max_val+1, size=(n, n))

    return matrix.astype(np.float32)
```

Программа, которая проверят приближенное равенство двух матриц.

Аргументы:

result\_strassen (np.array): Результат умножения матриц по алгоритму Штрассена.

result\_regular (np.array): Результат умножения матриц прямым методом.

tolerance (float): Допустимая погрешность при сравнении.

Возвращает:

bool: True, если матрицы приблизительно равны, иначе False.

```
def check_results(result_strassen, result_regular, tolerance = 1e-5):  
    return np.allclose(result_strassen, result_regular, atol=tolerance)  
  
if __name__ == '__main__':  
    n_values = [16, 32, 64]  
  
    for n in n_values:  
        mat_left = generate_square_matrix(n)  
        mat_right = generate_square_matrix(n)  
  
        result_strassen = strassen_multiply(mat_left, mat_right)  
        result_regular = matrix_multiply(mat_left, mat_right)  
  
        # Использование функции для проверки результатов  
        if check_results(result_strassen, result_regular):  
            print(f"Результаты умножения матриц - OK")  
        else:  
            print(f"Результаты умножения матриц НЕ совпадают")  
            exit(1)
```

Результаты умножения матриц - OK

Результаты умножения матриц - OK

Результаты умножения матриц - OK

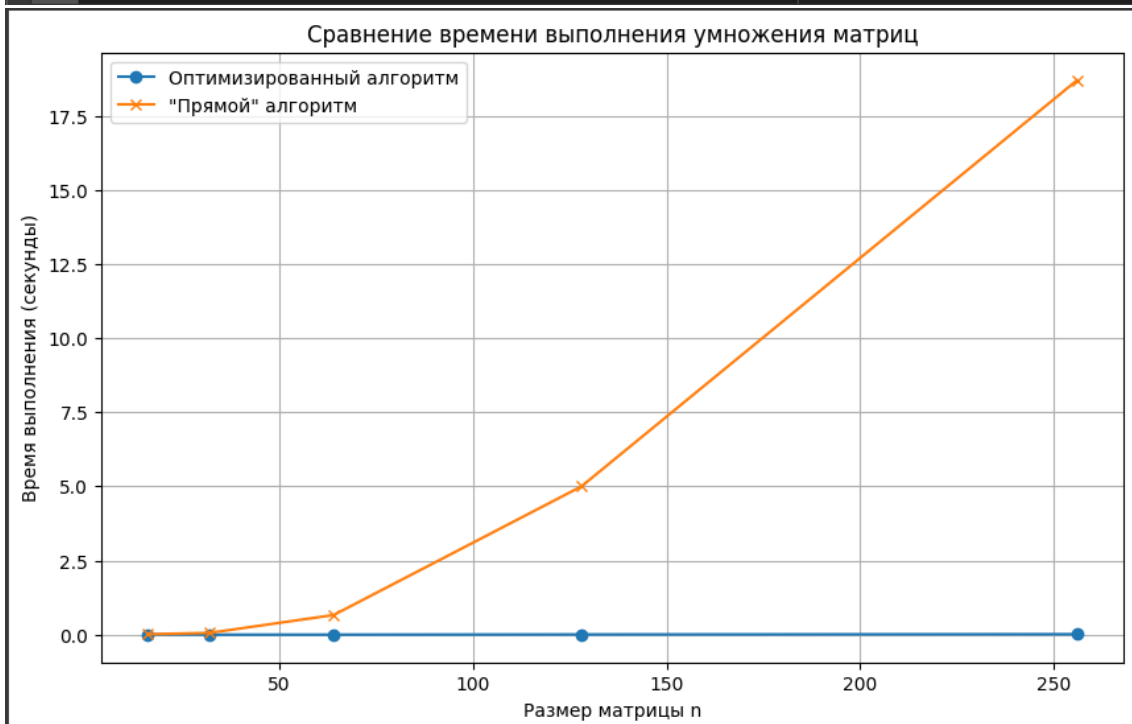
Количество операций умножения в разных методах

```
Результаты умножения матриц - OK  
6725600  
16777216
```

Первый результат для метода Штрассена и заметно что операций намного меньше, что доказывает его лучшую производительность

"Программы эмпирической оценки временной сложности функции dot для умножения матриц из библиотеки NumPy"

```
def optimize_mat_mul(mat_left, mat_right):  
    return np.dot(mat_left, mat_right)  
  
if __name__ == '__main__':  
  
    import matplotlib.pyplot as plt  
    import time  
  
    # Подготовка к замерам времени  
    n_values = [16, 32, 64, 128, 256]  
  
    opt_times = []  
    regular_times = []  
  
    for n in n_values:  
        mat_left = generate_square_matrix(n)  
        mat_right = generate_square_matrix(n)  
  
        # Замер времени для оптимизированного алгоритма  
        start_time_opt = time.time()  
        result_opt = optimize_mat_mul(mat_left, mat_right)  
        opt_times.append(time.time() - start_time_opt)  
  
        # Замер времени для прямого метода  
        start_time_regular = time.time()  
        result_regular = matrix_multiply(mat_left, mat_right)  
        regular_times.append(time.time() - start_time_regular)  
  
    # Построение графиков  
    plt.figure(figsize=(10, 6))  
    plt.plot(n_values, opt_times, label='Оптимизированный алгоритм', marker='o')  
    plt.plot(n_values, regular_times, label='Прямой алгоритм', marker='x')  
    plt.xlabel('Размер матрицы n')  
    plt.ylabel('Время выполнения (секунды)')  
    plt.title('Сравнение времени выполнения умножения матриц')  
    plt.legend()  
    plt.grid(True)  
    plt.show()
```



**Вывод:** прямой алгоритм работает за  $O(n^2)$  как видно по графику, ведь растёт параболой, из этого можно сделать заключение, что используемый алгоритм затратнее по сравнению с оптимизированным алгоритмом dot, что означает, что выгоднее использовать dot из библиотеки numpy для того чтобы умножать матрицы

"Программы для нахождения обратной матрицы."

```
import numpy as np

def compute_inverse_newton_schulz(matrix, num_iterations=5):
    assert matrix.shape[0] == matrix.shape[1]
    initial_approximation = matrix.T / (np.linalg.norm(matrix) ** 2)
    identity_matrix = np.eye(matrix.shape[0])

    for _ in range(num_iterations):
        initial_approximation = initial_approximation @ (2 * identity_matrix - matrix @ initial_approximation)

    return initial_approximation

matrix = np.array([[ 185.0, -174.0, 208.0],
                   [ 225.0, -193.0, 237.0],
                   [-217.0, 190.0, -239.0]], dtype = np.float64)

print("Исходная матрица:")
print(matrix)

# Вычисление обратной матрицы с использованием метода Ньютона-Шульца
inverse_matrix_ns = compute_inverse_newton_schulz(matrix, num_iterations=10)
print("\nОбратная матрица, полученная методом Ньютона-Шульца:")
print(inverse_matrix_ns)

# Вычисление обратной матрицы с использованием np.linalg.inv
inverse_matrix_np = np.linalg.inv(matrix)
print("\nОбратная матрица, полученная с помощью np.linalg.inv:")
print(inverse_matrix_np)

# Сравнение результатов
print("\nРазница между методами:")
print(inverse_matrix_ns - inverse_matrix_np)
```

```
Исходная матрица:
[[ 185. -174.  208.]
 [ 225. -193.  237.]
 [-217.  190. -239.]]

Обратная матрица, полученная методом Ньютона-Шульца:
[[-0.01194986  0.01210934  0.00024947]
 [-0.01296179  0.00404759 -0.00588386]
 [ 0.00174945 -0.00629048 -0.00652732]]

Обратная матрица, полученная с помощью np.linalg.inv:
[[-0.04476272  0.08430244  0.04464031]
 [-0.09572775 -0.0375811  -0.12057779]
 [-0.03545926 -0.10641857 -0.14057208]]

Разница между методами:
[[ 0.03281286 -0.0721931  -0.04439083]
 [ 0.08276596  0.04162869  0.11469394]
 [ 0.03720871  0.10012809  0.13404476]]
```

Вывод для этой программы : если сравнивать матрицу полученную методом Ньютона-Шульца с ручным решением, то можно заметить, что они одинаковые

При нахождение обратной матрицы с помощью np.linalg.inv выдаётся намного другой ответ, если посмотреть на разницу этих двух матриц

Программы для нахождения собственных чисел и собственных векторов матрицы.

```
[ ] def find_eigens(matrix):  
    eigenvalues, eigenvectors = np.linalg.eig(matrix)  
    return eigenvalues, eigenvectors  
  
A = np.array([[ 185.0, -174.0, 208.0],  
[ 225.0,-193.0, 237.0],  
[-217.0, 190.0, -239.0]], dtype=float)  
  
print("Исходная матрица:\n", A)  
eigenvalues, eigenvectors = find_eigens(A)  
print("Собственные числа матрицы:\n", eigenvalues)  
print("Собственные вектора матрицы:\n", eigenvectors)
```

Исходная матрица:

```
[[ 185. -174.  208.]  
 [ 225. -193.  237.]  
 [-217.  190. -239.]]
```

Собственные числа матрицы:

```
[-222.99447013 -17.84800059 -6.15752928]
```

Собственные вектора матрицы:

```
[[ 0.55249677  0.78137769 -0.53549711]  
 [ 0.58063361  0.20727389  0.33726462]  
 [-0.59800663 -0.58863099  0.7742709  ]]
```

Ссылка на блокнот с решением

[https://colab.research.google.com/drive/1mw9FahojM37-DwUz08L1\\_IA8pjyboUpi?usp=sharing](https://colab.research.google.com/drive/1mw9FahojM37-DwUz08L1_IA8pjyboUpi?usp=sharing)